Transformer Feed-Forward Layers Are Key-Value Memories

Mor Geva[1,2] **Roei Schuster**[1,3] **Jonathan Berant**[1,2] **Omer Levy**[1]
[1]Blavatnik School of Computer Science, Tel-Aviv University
[2]Allen Institute for Artificial Intelligence
[3]Cornell Tech
{morgeva@mail,joberant@cs,levyomer@cs}.tau.ac.il,rs864@cornell.edu

Abstract

Feed-forward layers constitute two-thirds of a transformer model's parameters, yet their role in the network remains under-explored. We show that feed-forward layers in transformer-based language models operate as key-value memories, where each key correlates with textual patterns in the training examples, and each value induces a distribution over the output vocabulary. Our experiments show that the learned patterns are human-interpretable, and that lower layers tend to capture shallow patterns, while upper layers learn more semantic ones. The values complement the keys' input patterns by inducing output distributions that concentrate probability mass on tokens likely to appear immediately after each pattern, particularly in the upper layers. Finally, we demonstrate that the output of a feed-forward layer is a composition of its memories, which is subsequently refined throughout the model's layers via residual connections to produce the final output distribution.

1 Introduction

Transformer-based language models (Vaswani et al., 2017) are at the core of state-of-the-art natural language processing (Devlin et al., 2019; Brown et al., 2020), largely due to the success of self-attention. While much literature has been devoted to analyzing the function of self-attention layers (Voita et al., 2019; Clark et al., 2019; Vig and Belinkov, 2019), they account for only a third of a typical transformer's parameters ($4d^2$ per layer, where d is the model's hidden dimension). Most of the parameter budget is spent on position-wise feed-forward layers ($8d^2$ per layer), yet their role remains under-explored. What, if so, is the function of feed-forward layers in a transformer language model?

We show that feed-forward layers emulate neural memories (Sukhbaatar et al., 2015), where the first parameter matrix in the layer corresponds to *keys*, and the second parameter matrix to *values*. Figure 1 shows how the keys (first parameter matrix) interact with the input to produce coefficients, which are then used to compute a weighted sum of the values (second parameter matrix) as the output. While the theoretical similarity between feed-forward layers and key-value memories has previously been suggested by Sukhbaatar et al. (2019), we take this observation one step further, and analyze the "memories" that the feed-forward layers store.

We find that each key correlates with a specific set of human-interpretable input patterns, such as n-grams or semantic topics. For example, k_2 in Figure 1 is triggered by inputs that describe a pe-

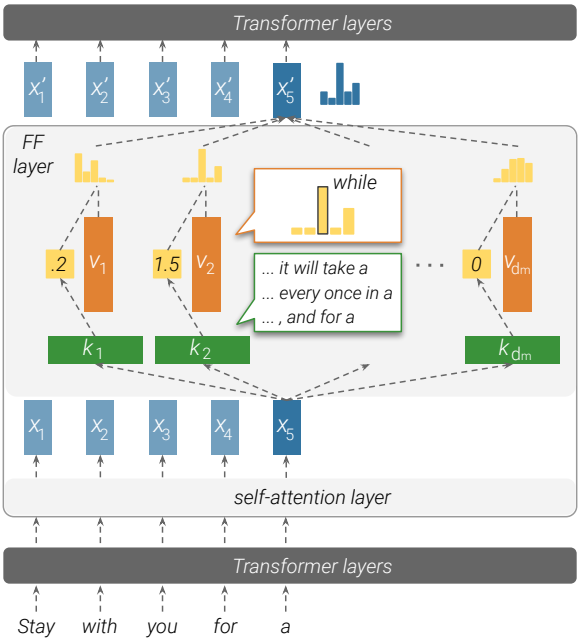

Figure 1: An illustration of how a feed-forward layer emulates a key-value memory. Input vectors (here, $\mathbf{x}_5$) are multiplied by *keys* to produce *memory coefficients* (e.g., the memory coefficient for $\mathbf{v}_1$ is 0.2), which then weigh distributions over the output vocabulary, stored in the *values*. The feed-forward layer's output is thus the weighted sum of its values.

riod of time and end with *"a"*. Simultaneously, we observe that each *value* can induce a distribution over the output vocabulary, and that this distribution correlates with the next-token distribution of the corresponding keys in the upper layers of the model. In the above example, the corresponding value v_2 represents a distribution that puts most of its probability mass on the word *"while"*.

Lastly, we analyze how the language model, as a whole, composes its final prediction from individual memories. We observe that each layer combines hundreds of active memories, creating a distribution that is qualitatively different from each of its component memories' values. Meanwhile, the residual connection between layers acts as a refinement mechanism, gently tuning the prediction at each layer while retaining most of the residual's information.

In conclusion, our work sheds light on the function of feed-forward layers in transformer-based language models. We show that feed-forward layers act as pattern detectors over the input across all layers, and that the final output distribution is gradually constructed in a bottom-up fashion.[1]

2 Feed-Forward Layers as Unnormalized Key-Value Memories

Feed-forward layers A transformer language model (Vaswani et al., 2017) is made of intertwined self-attention and feed-forward layers. Each feed-forward layer is a position-wise function, processing each input vector independently. Let $\mathbf{x} \in \mathbb{R}^d$ be a vector corresponding to some input text prefix. We can express the feed-forward layer $\text{FF}(\cdot)$ as follows (bias terms are omitted):

$$\text{FF}(\mathbf{x}) = f(\mathbf{x} \cdot K^\top) \cdot V \tag{1}$$

Here, $K, V \in \mathbb{R}^{d_m \times d}$ are parameter matrices, and f is a non-linearity such as ReLU.

Neural memory A neural memory (Sukhbaatar et al., 2015) consists of d_m key-value pairs, which we call *memories*.[2] Each key is represented by a d-dimensional vector $\mathbf{k}_i \in \mathbb{R}^d$, and together form the parameter matrix $K \in \mathbb{R}^{d_m \times d}$; likewise, we define the value parameters as $V \in \mathbb{R}^{d_m \times d}$. Given an input vector $\mathbf{x} \in \mathbb{R}^d$, we compute a distribution over the keys, and use it to compute the expected value:

$$p(k_i \mid x) \propto \exp(\mathbf{x} \cdot \mathbf{k}_i)$$
$$\text{MN}(\mathbf{x}) = \sum_{i=1}^{d_m} p(k_i \mid x)\mathbf{v}_i$$

With matrix notation, we arrive at a more compact formulation:

$$\text{MN}(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot K^\top) \cdot V \tag{2}$$

Feed-forward layers emulate neural memory Comparing equations 1 and 2 shows that feed-forward layers are almost identical to key-value neural memories; the only difference is that neural memory uses softmax as the non-linearity $f(\cdot)$, while the canonical transformer does not use a normalizing function in the feed-forward layer. The *hidden dimension* d_m is essentially the number of memories in the layer, and the activation $\mathbf{m} = f(\mathbf{x} \cdot K^\top)$, commonly referred to as the *hidden layer*, is a vector containing an unnormalized non-negative coefficient for each memory. We refer to each $\mathbf{m}_i$ as the *memory coefficient* of the ith memory cell.

Sukhbaatar et al. (2019) make an analogous observation, and incorporate the parameters of the feed-forward layers as persistent memory cells in the self-attention layers. While this reparameterization works in practice, the experiment does not tell us much about the role of feed-forward layers in the canonical transformer. If transformer feed-forward layers are indeed key-value memories, then what memories do they store?

We conjecture that each key vector $\mathbf{k}_i$ captures a particular pattern (or set of patterns) in the input sequence (Section 3), and that its corresponding value vector $\mathbf{v}_i$ represents the distribution of tokens that follows said pattern (Section 4).

3 Keys Capture Input Patterns

We posit that the key vectors K in feed-forward layers act as pattern detectors over the input sequence, where each individual key vector $\mathbf{k}_i$ corresponds to a specific pattern over the input prefix $x_1, \ldots, x_j$. To test our claim, we analyze the keys of a trained language model's feed-forward layers. We first retrieve the training examples (prefixes of a sentence) most associated with a given key, that is, the input texts where the memory coefficient is highest. We

[1] The code for reproducing our experiments is available at https://github.com/mega002/ff-layers/.

[2] We use the terms "memory cells" and "memories" interchangeably.

Key	Pattern	Example trigger prefixes
$\mathbf{k}^{1}_{449}$	Ends with *"substitutes"* (shallow)	*At the meeting, Elton said that "for artistic reasons there could be no substitutes* *In German service, they were used as substitutes* *Two weeks later, he came off the substitutes*
$\mathbf{k}^{6}_{2546}$	Military, ends with *"base"/"bases"* (shallow + semantic)	*On 1 April the SRSG authorised the SADF to leave their bases* *Aircraft from all four carriers attacked the Australian base* *Bombers flying missions to Rabaul and other Japanese bases*
$\mathbf{k}^{10}_{2997}$	a "part of" relation (semantic)	*In June 2012 she was named as one of the team that competed* *He was also a part of the Indian delegation* *Toy Story is also among the top ten in the BFI list of the 50 films you should*
$\mathbf{k}^{13}_{2989}$	Ends with a time range (semantic)	*Worldwide, most tornadoes occur in the late afternoon, between 3 pm and 7* *Weekend tolls are in effect from 7:00 pm Friday until* *The building is open to the public seven days a week, from 11:00 am to*
$\mathbf{k}^{16}_{1935}$	TV shows (semantic)	*Time shifting viewing added 57 percent to the episode's* *The first season set that the episode was included in was as part of the* *From the original NBC daytime version , archived*

Table 1: Examples of human-identified patterns that trigger different memory keys.

then ask humans to identify patterns within the retrieved examples. For almost every key $\mathbf{k}_i$ in our sample, a small set of well-defined patterns, recognizable by humans, covers most of the examples associated with the key.

3.1 Experiment

We conduct our experiment over the language model of Baevski and Auli (2019), a 16-layer transformer language model trained on WikiText-103 (Merity et al., 2017). This model defines $d = 1024$ and $d_m = 4096$, and has a total of $d_m \cdot 16 = 65,536$ potential keys to analyze. We randomly sample 10 keys per layer (160 in total).

Retrieving trigger examples We assume that patterns stored in memory cells originate from examples the model was trained on. Therefore, given a key $\mathbf{k}^{\ell}_{i}$ that corresponds to the i-th hidden dimension of the ℓ-th feed-forward layer, we compute the memory coefficient $\mathrm{ReLU}(\mathbf{x}^{\ell}_{j} \cdot \mathbf{k}^{\ell}_{i})$ for every prefix $x_1, \ldots, x_j$ of every sentence from the WikiText-103's training set.[3] For example, for the hypothetical sentence *"I love dogs"*, we will compute three coefficients, for the prefixes *"I"*, *"I love"*, and *"I love dogs"*. Then, we retrieve the *top-t trigger examples*, that is, the t prefixes whose representation at layer ℓ yielded the highest inner product with $\mathbf{k}^{\ell}_{i}$.

Pattern analysis We let human experts (NLP graduate students) annotate the top-25 prefixes retrieved for each key, and asked them to (a) identify repetitive patterns that occur in at least 3 prefixes (which would strongly indicate a connection to the key, as this would unlikely happen if sentences were drawn at random) (b) describe each recognized pattern, and (c) classify each recognized pattern as *"shallow"* (e.g. recurring n-grams) or *"semantic"* (recurring topic). Each key and its corresponding top-25 prefixes were annotated by one expert. To assure that every pattern is grounded in at least 3 prefixes, we instruct the experts to specify, for each of the top-25 prefixes, which pattern(s) it contains. A prefix may be associated with multiple (shallow or semantic) patterns.

Table 1 shows example patterns. A fully-annotated example of the top-25 prefixes from a single memory key is shown in Appendix A.

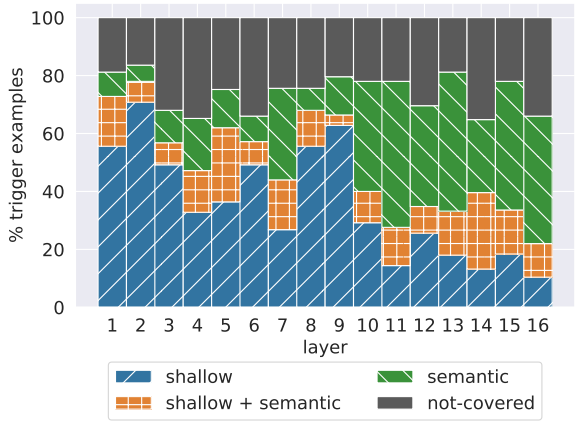

Figure 2: Breakdown of the labels experts assigned to trigger examples in each layer. Some examples were not associated with any pattern ("not-covered").

3.2 Results

Memories are associated with human-recognizable patterns Experts were able to identify at least one pattern for every key, with an average of 3.6 identified patterns per

[3] We segment training examples into sentences to simplify the annotation task and later analyses.

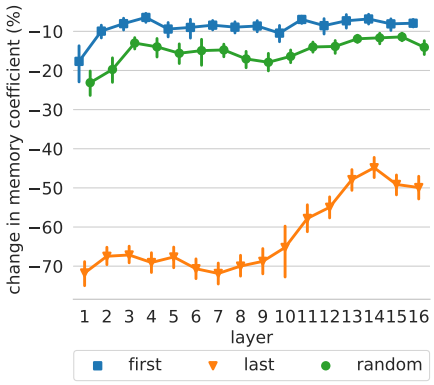

Figure 3: Relative change in memory coefficient caused by removing the first, the last, or a random token from the input.

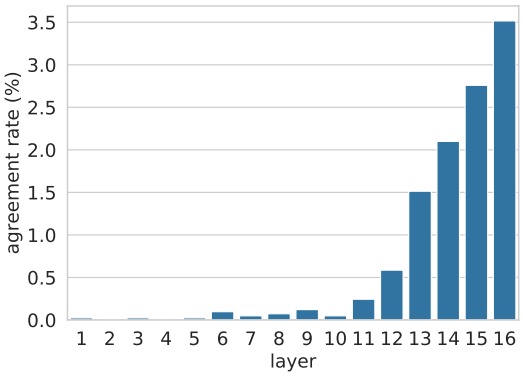

Figure 4: Agreement rate between the top-ranked token based on the value vector $\mathbf{v}_i^\ell$, and the next token of the top-ranked trigger example associated with the key vector $\mathbf{k}_i^\ell$.

key. Furthermore, the vast majority of retrieved prefixes (65%-80%) were associated with at least one identified pattern (Figure 2). Thus, the top examples triggering each key share clear patterns that humans can recognize.

Shallow layers detect shallow patterns Comparing the amount of prefixes associated with shallow patterns and semantic patterns (Figure 2), the lower layers (layers 1-9) are dominated by shallow patterns, often with prefixes that share the last word (e.g. $\mathbf{k}^1_{449}$ in Table 1). In contrast, the upper layers (layers 10-16) are characterized by more semantic patterns, with prefixes from similar contexts but without clear surface-form similarities (e.g. $\mathbf{k}^{16}_{1935}$ in Table 1). This observation corroborates recent findings that lower (upper) layers in deep contextualized models encode shallow (semantic) features of the inputs (Peters et al., 2018; Jawahar et al., 2019; Liu et al., 2019).

To further test this hypothesis, we sample 1600 random keys (100 keys per layer) and apply local modifications to the top-50 trigger examples of every key. Specifically, we remove either the *first*, *last*, or a *random* token from the input, and measure how this mutation affects the memory coefficient. Figure 3 shows that the model considers the end of an example as more salient than the beginning for predicting the next token. In upper layers, removing the last token has less impact, supporting our conclusion that upper-layer keys are less correlated with shallow patterns.

4 Values Represent Distributions

After establishing that keys capture patterns in training examples, we turn to analyze the information stored in their corresponding values. We show that each value $\mathbf{v}_i^\ell$ can be viewed as a distribution over the output vocabulary, and demonstrate that this distribution complements the patterns in the corresponding key $\mathbf{k}_i^\ell$ in the model's upper layers (see Figure 1).

Casting values as distributions over the vocabulary. We begin by converting each value vector $\mathbf{v}_i^\ell$ into a probability distribution over the vocabulary by multiplying it by the output embedding matrix E and applying a softmax:[4]

$$\mathbf{p}_i^\ell = \text{softmax}(\mathbf{v}_i^\ell \cdot E).$$

The probability distribution $\mathbf{p}_i^\ell$ is uncalibrated, since the value vector $\mathbf{v}_i^\ell$ is typically multiplied by the input-dependent memory coefficient $\mathbf{m}_i^\ell$, changing the skewness of the output distribution. That said, the *ranking* induced by $\mathbf{p}_i^\ell$ is invariant to the coefficient, and can still be examined. This conversion assumes (naïvely) that all model's layers operate in the same embedding space.

Value predictions follow key patterns in upper layers. For every layer ℓ and memory dimension i, we compare the top-ranked token according to $\mathbf{v}_i^\ell$, $(\text{argmax}(\mathbf{p}_i^\ell))$ to the next token w_i^ℓ in the top-1 trigger example according to $\mathbf{k}_i^\ell$ (the example whose memory coefficient for $\mathbf{k}_i^\ell$ is the highest). Figure 4 shows the *agreement rate*, i.e. the fraction of memory cells (dimensions) where the value's top prediction matches the key's top trigger example $(\text{argmax}(\mathbf{p}_i^\ell) = w_i^\ell)$. It can be seen that the

[4]This is a simplification; in practice, we use the adaptive softmax (Baevski and Auli, 2019) to compute probabilities.